# DATA QUALITY AUDIT REPORT

*SaaS Operations & Churn Analysis Project*

Decision to Replace Source Dataset

| Project | Date | Decision |
|---|---|---|
| **SaaS Churn Analysis** | **April 2026** | **Dataset Replaced** |

## 1. Executive Summary

**Decision**

The original SaaS dataset was rejected and replaced following a structured data quality audit that revealed a fundamental data generation flaw. 13,198 of 25,000 feature usage records (52.8%) predate the account signup date of the associated account — a logical impossibility in a real production system. Investigation confirmed this is a structural constraint violation in how the synthetic dataset was generated, not a fixable data quality issue. Proceeding with this dataset would produce unreliable retention, cohort, and churn metrics.

## 2. Original Dataset Overview

The original dataset was intended to represent a SaaS business with the following structure:

| Table | Records | Description |
|---|---|---|
| accounts | 500 | Account profiles with signup dates |
| subscriptions | ~500+ | Subscription plans with MRR and dates |
| feature_usage | 25,000 | Feature interaction events with timestamps |
| support_tickets | 2000 | Customer support interactions |
| churn_events | 600 | Churn records linked to accounts |

The following date integrity validators were executed against all tables to identify temporal anomalies:

| Check | Description | Result |
|---|---|---|
| Check 1 | Subscriptions where end_date < start_date | 0 violations |
| Check 2 | Support tickets closed before submitted | 0 violations |
| Check 3 | Feature usage before account signup date | 13,198 violations |
| Check 4 | Subscriptions starting before account signup | 0 violations |
| Check 5 | Churn events before account signup | 0 violations |

**Key Observation**

Checks 1, 2, 4, and 5 returned zero violations because they compare dates within the same system or a tightly coupled relationship — the application enforces order naturally. Check 3 crosses three separate tables (feature_usage → subscriptions → accounts), which exposed a constraint that was never enforced during data generation.

# 3. Investigation — Five-Layer Analysis

Each finding below ruled out a specific hypothesis, progressively narrowing the root cause to a definitive conclusion.

## 3.1 Initial Detection

The following query identified all feature usage records where the usage date precedes the associated account's signup date:

```
SELECT f.usage_id, f.usage_date, a.signup_date
FROM feature_usage f
JOIN subscriptions s ON f.subscription_id = s.subscription_id
JOIN accounts a      ON s.account_id = a.account_id
WHERE f.usage_date < a.signup_date;
```

| Metric | Value |
|---|---|
| Total feature_usage records | 25,000 |
| Records where usage_date < signup_date | 13,198 |
| Percentage of dataset affected | 52.8% |
| Unique accounts affected | 490 of 500 (98%) |

## 3.2 Hypothesis 1 — Clock Skew Between Systems

Clock skew occurs when two systems recording timestamps operate in different time zones or with unsynchronised clocks, causing millisecond-to-second offsets. This was tested by analysing the size of the time gap between usage_date and signup_date across all violations.

```
SELECT
    CASE
        WHEN (a.signup_date - f.usage_date) < 1   THEN '< 1 day'
        WHEN (a.signup_date - f.usage_date) < 7   THEN '1-7 days'
        WHEN (a.signup_date - f.usage_date) < 30  THEN '7-30 days'
        WHEN (a.signup_date - f.usage_date) < 90  THEN '30-90 days'
        ELSE '90+ days'
    END AS gap_bucket,
    COUNT(*)                                                    AS records,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1)         AS pct
FROM feature_usage f
JOIN subscriptions s ON f.subscription_id = s.subscription_id
JOIN accounts a      ON s.account_id = a.account_id
WHERE f.usage_date < a.signup_date
```

```
GROUP BY 1
ORDER BY MIN(a.signup_date - f.usage_date);
```

| Gap Bucket | Records | Percentage |
|---|---|---|
| < 1 day | 0 | 0.0% |
| 1–7 days | 208 | 1.6% |
| 7–30 days | 833 | 6.3% |
| 30–90 days | 1,892 | 14.3% |
| 90+ days | 10,265 | 77.8% |

**Hypothesis 1 Rejected**

Clock skew produces gaps measured in seconds or minutes — not days. Zero records fall under the < 1 day threshold. 77.8% of violations exceed 90 days. Clock skew is definitively ruled out.

## 3.3 Hypothesis 2 — Legitimate Freemium / Trial Usage

Many SaaS products allow users to interact with the product before completing formal account registration. Under this hypothesis, pre-signup usage would be concentrated in the 1–30 day range (a typical trial window), with counts decreasing at longer gaps.

Expected distribution under a freemium hypothesis:

| Gap Bucket | Expected Pattern | Actual Result |
|---|---|---|
| < 1 day | Low — same-session signups only | 0 records |
| 1–7 days | High — peak trial activity | 208 (1.6%) |
| 7–30 days | High — typical trial window | 833 (6.3%) |
| 30–90 days | Declining — extended evaluations | 1,892 (14.3%) |
| 90+ days | Near zero — beyond any trial | 10,265 (77.8%) |

**Hypothesis 2 Rejected**

The actual distribution is the inverse of what a freemium model would produce. The largest bucket is 90+ days — well beyond any realistic trial or evaluation period. No SaaS product has a 3-month pre-signup trial as standard behaviour. Freemium usage is definitively ruled out.

## 3.4 Hypothesis 3 — Data Migration Event

A one-time migration event can overwrite signup_date values for a batch of accounts with a single new date, making historical usage appear to precede signup. This would produce a spike in violations concentrated around one specific month.

The violation count was grouped by signup month to look for this spike:

```
SELECT
    DATE_TRUNC('month', a.signup_date)   AS signup_month,
    COUNT(DISTINCT a.account_id)          AS accounts,
    MIN(f.usage_date)                     AS earliest_usage,
    MAX(f.usage_date)                     AS latest_usage,
    ROUND(AVG(a.signup_date - f.usage_date)) AS avg_gap_days
FROM feature_usage f
JOIN subscriptions s ON f.subscription_id = s.subscription_id
JOIN accounts a      ON s.account_id = a.account_id
WHERE f.usage_date < a.signup_date
GROUP BY 1
ORDER BY 1 DESC;
```

| Signup Month | Accounts | Earliest Usage | Latest Usage | Avg Gap (Days) |
|---|---|---|---|---|
| Dec 2024 | 17 | 2023-01-01 | 2024-12-26 | 362 |
| Nov 2024 | 32 | 2023-01-01 | 2024-11-27 | 347 |
| Oct 2024 | 31 | 2023-01-01 | 2024-10-29 | 325 |
| ... | ... | ... | ... | ... |
| Mar 2023 | 20 | 2023-01-01 | 2023-03-27 | 40 |
| Feb 2023 | 17 | 2023-01-02 | 2023-02-26 | 25 |
| Jan 2023 | 8 | 2023-01-03 | 2023-01-26 | 15 |

**Critical Finding**

Two patterns immediately rule out a migration event: (1) The earliest_usage column shows 2023-01-01 for every signup cohort — including accounts that signed up in late 2024. (2) The avg_gap_days grows linearly with each month, which is consistent with random seeding from a fixed start date, not a migration. A migration spike would show one outlier month with near-zero violations elsewhere.

**Hypothesis 3 Rejected**

No migration spike exists. Violations are distributed consistently across all 24 months with a steady linear increase. The earliest_usage pinned at 2023-01-01 across all cohorts is

inconsistent with a migration event and points to random date generation from a fixed dataset start date.

## 3.5 Root Cause Confirmation — Account-Level Audit

A final account-level query compared the signup date distribution of affected vs. unaffected accounts to determine the scope and nature of the problem:

```
WITH affected AS (
    SELECT DISTINCT a.account_id
    FROM feature_usage f
    JOIN subscriptions s ON f.subscription_id = s.subscription_id
    JOIN accounts a      ON s.account_id = a.account_id
    WHERE f.usage_date < a.signup_date
)
SELECT
    CASE
        WHEN a.account_id IN (SELECT account_id FROM affected)
        THEN 'Affected accounts'
        ELSE 'Clean accounts'
    END                                                          AS account_group,
    COUNT(*)                                                     AS
total_accounts,
    MIN(a.signup_date)                                           AS
earliest_signup,
    MAX(a.signup_date)                                           AS latest_signup,
    TO_TIMESTAMP(AVG(EXTRACT(EPOCH FROM a.signup_date::timestamptz)))::date
                                                                 AS
avg_signup_date
FROM accounts a
GROUP BY 1;
```

| Account Group | Total Accounts | Earliest Signup | Latest Signup | Avg Signup Date |
|---|---|---|---|---|
| Affected accounts | 490 | 2023-01-20 | 2024-12-31 | 2024-02-03 |
| Clean accounts | 10 | 2023-01-02 | 2023-02-09 | 2023-01-16 |

This result is definitive. Three observations confirm the root cause:

- 98% of accounts (490 of 500) are affected — this is a dataset-wide structural problem, not an isolated data quality issue.
- The 10 clean accounts all signed up within the first 6 weeks of the dataset (January 2 to February 9, 2023). They escaped the violation only because their signup date was so close to the dataset start date (2023-01-01) that there was almost no window for randomly generated usage to land before it.
- The avg_gap_days growing proportionally with calendar distance from January 2023 is the mathematical fingerprint of random generation from a fixed start date without a date constraint.

**Root Cause Confirmed**

Usage dates in the feature_usage table were randomly generated across the full dataset date range (2023-01-01 to 2024-12-31) without enforcing the constraint that usage_date must be greater than or equal to the account's signup_date. This is a data generation flaw, not a real-world data quality issue that can be resolved through filtering or imputation.

# 4. Impact Assessment

The following table summarises the downstream impact of the constraint violation on each planned analytical deliverable:

| Analysis | Severity | Impact Description |
|---|---|---|
| **Cohort Retention Analysis** | **Critical** | Only 10 of 500 accounts are unaffected. Cohort sizes of ~1 account per month make retention curves statistically meaningless. |
| **Monthly Churn Rate** | **Critical** | Churn rate is a ratio. Denominators of 1–2 accounts per cohort produce wild percentage swings that do not represent real churn behaviour. |
| **Feature Adoption Metrics** | **High** | 52.8% of usage records must be excluded. Any 'most-used feature' or 'features correlated with churn' finding is based on less than half the intended data. |
| **Time-to-First-Value Analysis** | **Critical** | Days-to-first-use after signup is entirely corrupted. 98% of accounts show usage before they signed up, making this metric impossible to calculate. |
| **MRR Movement (Expansion/Contraction)** | **Low** | MRR data is date-independent for snapshot calculations. Less affected but cohort-based MRR analysis still unreliable. |
| **Churn Prediction Model** | **High** | Pre-signup usage artificially inflates early engagement signals, biasing feature importance scores in any predictive model. |

Why filtering alone is not sufficient:

- Applying WHERE f.usage_date >= a.signup_date removes 13,198 records, leaving 11,802 — only 47% of the original dataset.
- With 98% of accounts affected, excluding pre-signup records does not produce a clean subset — it produces a severely depleted dataset where most accounts have minimal remaining usage history.
- Cohort analysis requires adequate sample sizes per time period. After filtering, average cohort size drops below 1 account per month — insufficient for any reliable statistical analysis.
- The root cause is in data generation, not data collection. No transformation or imputation strategy can reconstruct what the correct usage dates should have been.

# 5. Decision & Justification

**Formal Decision: Replace Source Dataset**

The original dataset is retired from use in this project. A new dataset will be sourced or synthetically generated with proper temporal constraints enforced at the point of generation. The data quality audit documentation is retained as Section 1 of the project portfolio.

## 5.1 Justification

The decision to replace rather than repair the dataset is based on three criteria:

| Criterion | Assessment | Conclusion |
|---|---|---|
| Repairability | Root cause is in generation logic, not collection. No repair strategy can reconstruct correct usage timestamps. | Cannot be repaired |
| Filtered dataset viability | Post-filter dataset retains only 47% of records. 98% of accounts affected means no clean cohort exists. | Not viable for analysis |
| Analysis reliability | All core deliverables (cohort retention, churn rate, feature adoption) are critically impacted. | Results would be unreliable |

## 5.2 What Is Retained From This Dataset

The investigative work conducted on this dataset has standalone portfolio value and is retained in the project documentation:

- Complete date integrity validation framework — 5 checks covering all table relationships
- MD5 hash-based duplicate classification methodology (exact duplicates vs. ID collisions)
- Five-layer root cause analysis process — a reusable template for any data quality investigation
- Gap distribution analysis technique for diagnosing temporal anomalies
- Account-level cohort auditing approach

# 6. Replacement Dataset Requirements

The replacement dataset must satisfy the following constraints to support a credible SaaS churn analysis:

## 6.1 Mandatory Temporal Constraints

- feature_usage.usage_date >= accounts.signup_date for all records
- subscriptions.start_date >= accounts.signup_date for all records
- churn_events.churn_date > accounts.signup_date for all records
- No end_date < start_date in any table

## 6.2 Minimum Scale Requirements

| Dimension | Minimum Requirement | Rationale |
|---|---|---|
| Total accounts | 400–600 | Sufficient for monthly cohorts of 30–50 accounts |
| Feature usage records | 20,000+ | Enough for per-feature adoption analysis |
| Date range | 18–24 months | Required for long-term retention curve visibility |
| Churn rate | 15–35% | Realistic SaaS range — enough churned accounts to model |

## 6.3 Required Fields

The following fields are necessary to support the planned analytical deliverables:

| Table | Required Fields |
|---|---|
| accounts | account_id, signup_date, plan_type, industry, company_size |
| subscriptions | subscription_id, account_id, mrr, start_date, end_date, plan |
| feature_usage | usage_id, account_id, feature_name, usage_date, usage_count |
| churn_events | event_id, account_id, churn_date, churn_reason |

## 6.4 Recommended Data Sources

- Kaggle — Telco Customer Churn (IBM): 7,043 records, clean structure, widely validated
- Kaggle — SaaS Sales / B2B Churn datasets: MRR-based with subscription tier data
- Synthetically generated dataset using Python with all temporal constraints enforced at generation time (preferred — enables full control of schema and narrative)

# 7. Appendix — Full SQL Audit Trail

All queries used in this investigation are documented below for reproducibility.

### A. Date Integrity Validators

```
-- Check 1: Subscriptions ending before they start
SELECT subscription_id, start_date, end_date
FROM subscriptions
WHERE end_date < start_date;

-- Check 2: Support tickets closed before submitted
SELECT ticket_id, submitted_at, closed_at
FROM support_tickets
WHERE closed_at < submitted_at;

-- Check 3: Feature usage before account signup (returned 13,198 violations)
SELECT f.usage_id, f.usage_date, a.signup_date
FROM feature_usage f
JOIN subscriptions s ON f.subscription_id = s.subscription_id
JOIN accounts a      ON s.account_id = a.account_id
WHERE f.usage_date < a.signup_date;

-- Check 4: Subscriptions starting before account signup
SELECT s.subscription_id, s.start_date, a.signup_date
FROM subscriptions s
JOIN accounts a ON s.account_id = a.account_id
WHERE s.start_date < a.signup_date;

-- Check 5: Churn events before account signup
SELECT c.account_id, c.churn_date, a.signup_date
FROM churn_events c
JOIN accounts a ON c.account_id = a.account_id
WHERE c.churn_date < a.signup_date;
```

### B. Duplicate Detection — Hash-Based Classification

```
SELECT
    usage_id,
    COUNT(*)                                                        AS
total_rows,
    COUNT(DISTINCT md5(CAST(
        (usage_date, feature_name, subscription_id, usage_count)
        AS TEXT)))                                                  AS
distinct_hashes,
    CASE
        WHEN COUNT(DISTINCT md5(CAST(
            (usage_date, feature_name, subscription_id, usage_count)
            AS TEXT))) = 1 THEN 'Exact duplicate'
```

```
        ELSE 'ID collision'
    END                                                                 AS
duplicate_type
FROM feature_usage
GROUP BY usage_id
HAVING COUNT(*) > 1
ORDER BY duplicate_type, usage_id;
```

### C. Gap Distribution Analysis

```
SELECT
    CASE
        WHEN (a.signup_date - f.usage_date) < 1  THEN '< 1 day'
        WHEN (a.signup_date - f.usage_date) < 7  THEN '1-7 days'
        WHEN (a.signup_date - f.usage_date) < 30 THEN '7-30 days'
        WHEN (a.signup_date - f.usage_date) < 90 THEN '30-90 days'
        ELSE '90+ days'
    END                                                    AS gap_bucket,
    COUNT(*)                                               AS records,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 1)    AS pct
FROM feature_usage f
JOIN subscriptions s ON f.subscription_id = s.subscription_id
JOIN accounts a      ON s.account_id = a.account_id
WHERE f.usage_date < a.signup_date
GROUP BY 1
ORDER BY MIN(a.signup_date - f.usage_date);
```

### D. Signup Month Cohort Audit

```
SELECT
    DATE_TRUNC('month', a.signup_date)              AS signup_month,
    COUNT(DISTINCT a.account_id)                    AS accounts,
    MIN(f.usage_date)                               AS earliest_usage,
    MAX(f.usage_date)                               AS latest_usage,
    ROUND(AVG(a.signup_date - f.usage_date))        AS avg_gap_days
FROM feature_usage f
JOIN subscriptions s ON f.subscription_id = s.subscription_id
JOIN accounts a      ON s.account_id = a.account_id
WHERE f.usage_date < a.signup_date
GROUP BY 1
ORDER BY 1 DESC;
```

### E. Affected vs. Clean Account Comparison

```
WITH affected AS (
    SELECT DISTINCT a.account_id
    FROM feature_usage f
    JOIN subscriptions s ON f.subscription_id = s.subscription_id
```

```
    JOIN accounts a      ON s.account_id = a.account_id
    WHERE f.usage_date < a.signup_date
)
SELECT
    CASE
        WHEN a.account_id IN (SELECT account_id FROM affected)
        THEN 'Affected accounts'
        ELSE 'Clean accounts'
    END                                                          AS
account_group,
    COUNT(*)                                                     AS
total_accounts,
    MIN(a.signup_date)                                           AS
earliest_signup,
    MAX(a.signup_date)                                           AS
latest_signup,
    TO_TIMESTAMP(
        AVG(EXTRACT(EPOCH FROM a.signup_date::timestamptz))
    )::date                                                      AS
avg_signup_date
FROM accounts a
GROUP BY 1;
```

**Data Quality Audit Complete**

*This document supports the decision to replace the original SaaS dataset.*